



What is a Hash?

- A hash is a fixed-size output produced by a hash function from any input.
- It's used to verify data integrity and to store passwords safely (store hash, not plain password).
- Hashes are deterministic: same input → same hash every time.

Hash properties

- **Deterministic:** same input always gives the same hash.
- **Fixed length:** output size is constant (e.g., 256 bits) regardless of input size.
- **One-way:** infeasible to recover original input from the hash.
- **Avalanche effect:** tiny input change produces a vastly different hash.



Hash Types

- **MD5:** fast, broken for collision resistance, not recommended for security.
- **SHA-1:** historically popular, now deprecated for collision resistance.
- **SHA-256 / SHA-3:** modern cryptographic hashes for integrity (SHA-256 common).
- **bcrypt, scrypt, Argon2:** password hashing functions designed to be slow and memory-hard (better for storing passwords).



Salting & Peppering

Why they matter?

- **Salt:** a unique random value per password stored with the hash. It prevents reuse attacks and rainbow table lookups.
- **Pepper:** a secret global value stored separately (e.g., in application config or HSM). It adds extra secrecy if DB leaks.
- **Why:** salts make identical passwords produce different hashes; pepper adds another obstacle for attackers.

Common attacks on hashes

- **Rainbow tables:** precomputed hash→password tables (defeated by salts).
- **Brute-force:** trying all possible passwords (cost depends on hash speed / complexity).
- **Dictionary attack:** trying common passwords from wordlists.
- **Collision attacks:** finding two inputs with same hash (important for MD5/SHA-1).

 Difference:
Offline vs Online attacks

- **Offline attacks:** attacker has the hash database and can try guesses locally (fast if hash function is fast).
- **Online attacks:** attacker attempts logins against a live system (throttled/rate-limited, IP/behavior monitored).
- **Key point:** offline attacks are more dangerous. They can be massively parallelized unless hashes are slow/strong.

 **Tools:**

(Hydra, John the Ripper, Hashcat)



Best Practices & Defenses

- Use slow, memory-hard password hashes (Argon2, scrypt, bcrypt) with strong parameters.
- Store unique random salts per password and keep peppers secret (in HSM if possible).
- Implement rate limiting, account lockouts, and multi-factor authentication (MFA) to reduce online risk.
- Monitor for breaches, rotate secret keys, and enforce strong password policies (length & entropy).
- Prefer hashing on the server side, with proper config, and protect backups/DB access.